

Averages and Spread

Mean, standard deviation, and keeping numbers tame · video 1:56

This lesson has one big idea: two simple statistics describe a list of numbers, the **mean** (its center) and the **standard deviation** (its spread), and together they let us keep numbers tame. Normalizing with them is what layer norm does, and the same idea of spread explains why attention divides its scores by $\sqrt{d}$.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 8).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The mean: the center	▶ 0:08-0:27	4	☐
2. Standard deviation: the typical distance	▶ 0:28-0:51	4	☐
3. Normalizing: mean 0, spread 1	▶ 0:52-1:14	4	☐
4. The spread of a sum grows like $\sqrt{n}$	▶ 1:14-1:25	4	☐
5. Dividing by $\sqrt{d}$ keeps softmax tame	▶ 1:26-1:43	3	☐

Before you start: concepts 1–3 need only arithmetic and square roots. Concepts 4 and 5 use the dot product from F02 (multiply matching numbers, then add) and softmax from F07 (bigger gaps between scores make it sharper). These ideas are used in episode 6 (the $\sqrt{d}$ in the attention formula) and episode 9 (layer norm in the transformer block).

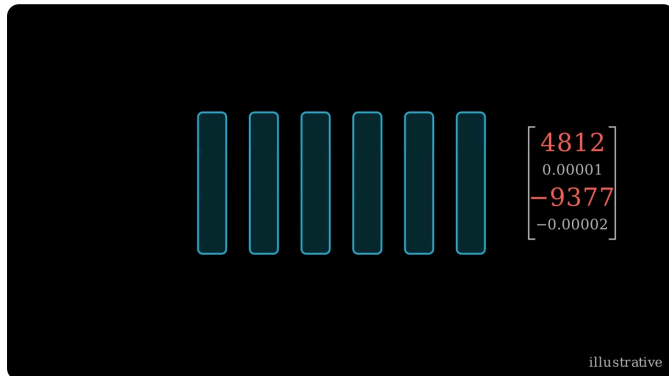
CONCEPT 1 OF 5

The mean: the center

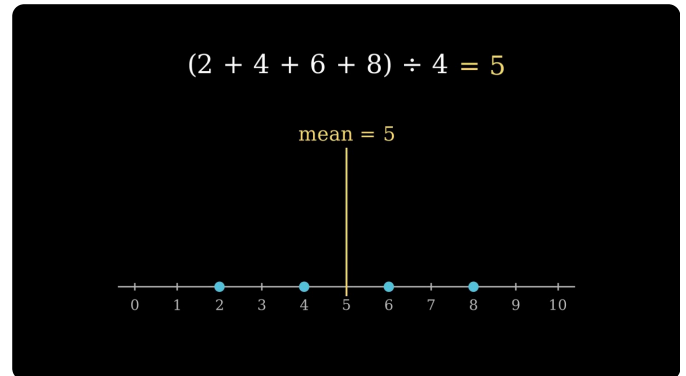
▶ Video 0:08-0:27

Deep networks pass numbers through dozens of layers. If they drift too big (4812) or too small (0.00001), training breaks. Two simple statistics help keep them in check: the **mean**, which says where the numbers are centered, and the **standard deviation**, which says how spread out they are.

The **mean** is the average: add the numbers up and divide by how many there are. For 2, 4, 6, 8: $(2 + 4 + 6 + 8) \div 4 = 20 \div 4 = 5$.



▶ 0:15 Through many layers, numbers can drift too big or too small.



▶ 0:27 The mean of 2, 4, 6, 8 is 5, the center of the dots.

KEY IDEA

mean = sum ÷ count. It tells you where the numbers are centered, but nothing about how spread out they are.

Check 1: The mean: the center

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

Find the mean of **1, 2, 3, 10**.

.....

1.2 CALCULATION

Add 10 to every number of the video's example: 12, 14, 16, 18. What is the new mean?

.....

1.3 TRUE OR FALSE

"Two lists with the same mean must be spread out in the same way."

True ☐ False ☐

1.4 MULTIPLE CHOICE

Why do deep networks need to keep their numbers in check?

- A.** Numbers that drift too big or too small through many layers make training break
- B.** Every layer's mean must be exactly 5
- C.** Negative numbers are not allowed inside a network
- D.** Computers can only store numbers between 0 and 1

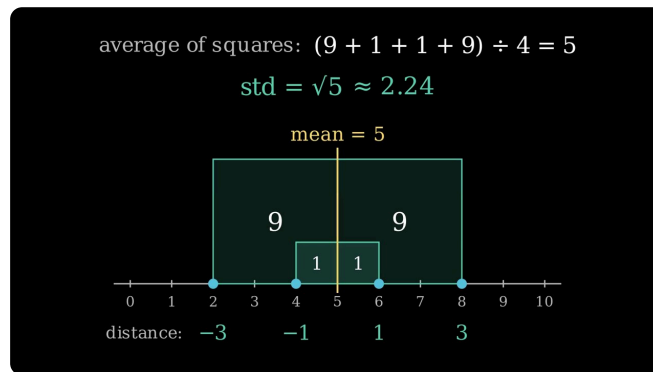
CONCEPT 2 OF 5

Standard deviation: the typical distance

▶ Video 0:28-0:51

The **standard deviation** (std) measures spread: how far the numbers typically are from the mean. The recipe: take each number's **distance from the mean**, **square** it, **average** the squares, and take the **square root**.

For 2, 4, 6, 8 (mean 5): the distances are $-3, -1, 1, 3$; the squares are 9, 1, 1, 9; their average is $20 \div 4 = 5$; so the std is $\sqrt{5} \approx 2.24$. Squaring makes every distance count as positive (-3 and 3 both give 9); the square root undoes the squaring.



▶ 0:50 Squares 9, 1, 1, 9; average 5; std = $\sqrt{5} \approx 2.24$.

KEY IDEA

std = $\sqrt{\text{average of the squared distances from the mean}}$. A small std means the numbers huddle close to the mean; a large std means they are spread out.

Check 2: Standard deviation: the typical distance

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 PUT IN ORDER

Put the steps of the std recipe in order: *square · take the square root · distance from the mean · average*.

2.2 CALCULATION

Find the std of **1, 2, 3, 4, 5**.

2.3 MULTIPLE CHOICE

What is the std of **5, 5, 5, 5**?

- A. 5
- B. 1
- C. 0
- D. 2.24

2.4 TRUE OR FALSE

"Doubling every number (4, 8, 12, 16) doubles the std."

True ☐ False ☐

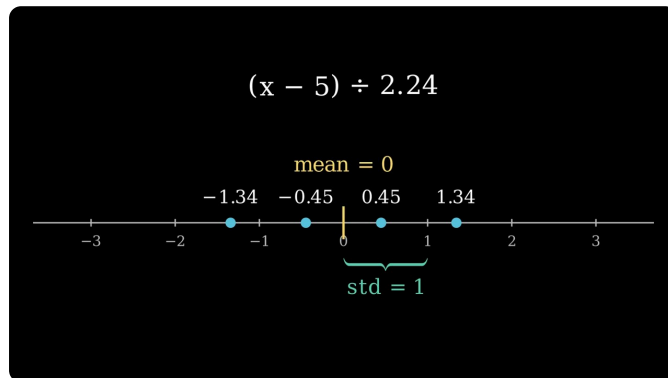
CONCEPT 3 OF 5

Normalizing: mean 0, spread 1

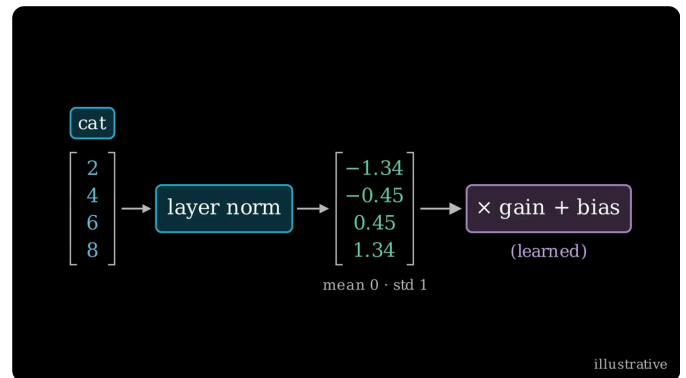
▶ Video 0:52-1:14

To **normalize**, subtract the mean, then divide by the std. For 2, 4, 6, 8: $(x - 5) \div 2.24$ gives about **-1.34, -0.45, 0.45, 1.34**. The mean is now 0 and the spread is 1, whatever the numbers were before.

That is exactly what **layer norm** does to every token's vector, followed by a learned **scale and shift** ($\times \text{gain} + \text{bias}$): the network itself learns which center and spread work best.



▶ 1:08 $(x - 5) \div 2.24$: the numbers now have mean 0 and std 1.



▶ 1:13 Layer norm does this to every token's vector, then $\times \text{gain} + \text{bias}$.

KEY IDEA

Normalize: $(x - \text{mean}) \div \text{std} \rightarrow \text{mean } 0, \text{std } 1$. Layer norm = normalize each token's vector, then $\times \text{gain} + \text{bias}$ (both learned).

Check 3: Normalizing: mean 0, spread 1

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

Normalize the two numbers **1, 3**.

3.2 MULTIPLE CHOICE

Which list normalizes to exactly the same numbers as 2, 4, 6, 8 (-1.34, -0.45, 0.45, 1.34)?

- A. 10, 20, 30, 40
- B. 2, 4, 6, 9
- C. 1, 2, 3
- D. 2, 4, 6, 8, 10

3.3 CALCULATION

Layer norm has turned a vector into -1, 1. The learned gain is 2 and the bias is 1. What comes out? What are its mean and std?

3.4 TRUE OR FALSE

"After normalizing, every number lies between -1 and 1."

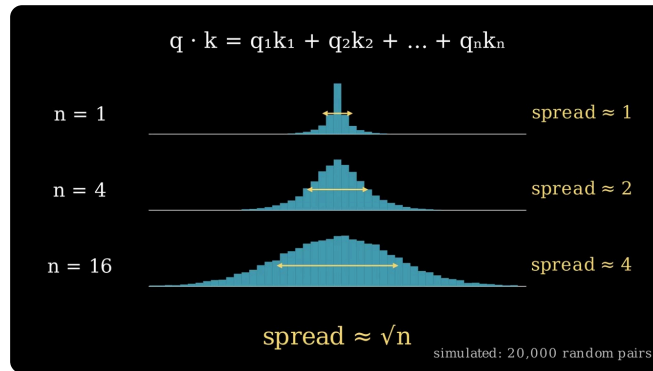
True ☐ False ☐

CONCEPT 4 OF 5

The spread of a sum grows like $\sqrt{n}$

▶ Video 1:14-1:25

Spread also explains a detail in attention. A dot product $q \cdot k = q_1k_1 + q_2k_2 + \dots + q_nk_n$ adds up **n products**. When the numbers are random, some products are positive and some negative, so they partly cancel. The spread of the total still grows, but only with the **square root** of how many you add. The video simulated 20,000 random pairs (numbers with mean 0 and spread 1): $n = 1$ gives a spread of about 1, $n = 4$ about 2, $n = 16$ about 4.



▶ 1:24 Simulated dot products: 4 times the terms, twice the spread.

KEY IDEA

Add up n random products and the spread of the total is about $\sqrt{n}$. Four times as many terms only doubles the spread.

Check 4: The spread of a sum grows like $\sqrt{n}$

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

Following the pattern, what spread do you expect for $n = 64$? For $n = 256$?

4.2 MULTIPLE CHOICE

Going from $n = 16$ to $n = 64$ terms multiplies the spread by about...

- A. 4
- B. 2
- C. 8
- D. 16

4.3 TRUE OR FALSE

"Add up 100 random products, each with a spread of about 1, and the total has a spread of about 100."

True ☐ False ☐

4.4 CALCULATION

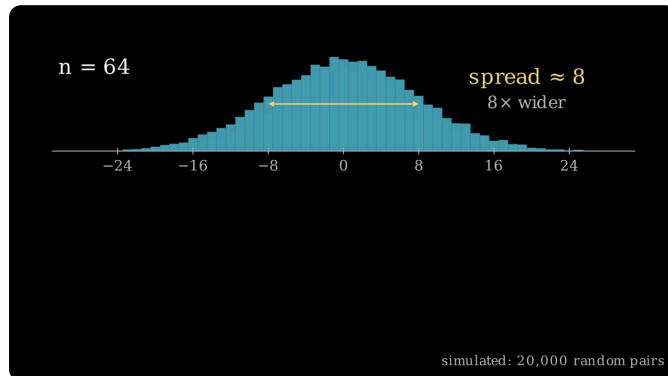
GPT-2's vectors have 768 numbers. If you took the dot product of two random vectors that size, about how wide would the spread be?

CONCEPT 5 OF 5

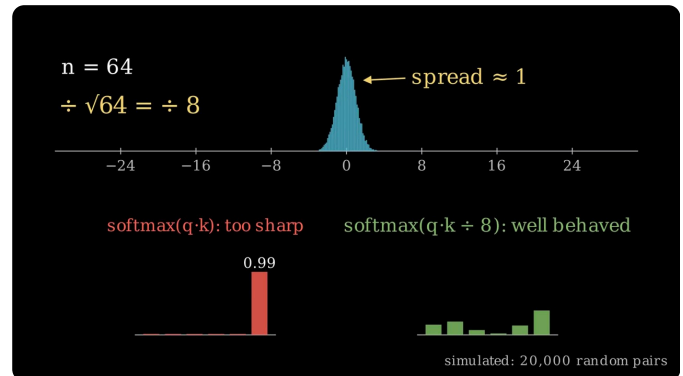
Dividing by $\sqrt{d}$ keeps softmax tame

▶ Video 1:26-1:43

With vectors of 64 numbers, dot products spread about $\sqrt{64} = 8$ times wider. Fed straight into softmax, such big gaps make it far too sharp: in the video, one key gets 0.99 and the others almost nothing. Dividing by the square root of the key size, $\sqrt{d}$, brings the spread back to about 1, which keeps the softmax well behaved.



▶ 1:30 With 64 numbers, dot products spread about 8 times wider.



▶ 1:37 $\div \sqrt{64} = \div 8$: spread about 1, and a well-behaved softmax.

In NumPy, mean and std are built in, and normalizing is one line:

```
x = np.array([2, 4, 6, 8])
x.mean(), x.std()           # (5.0, 2.236)
(x - x.mean()) / x.std()    # [-1.34, -0.45, 0.45, 1.34]

q, k = np.random.randn(2, 64)
q @ k / np.sqrt(64)         # typically around ±1
```

KEY IDEA

Attention divides its scores by $\sqrt{d}$. That cancels the $\sqrt{d}$ growth of the spread, so softmax sees scores with a spread of about 1 and stays well behaved, not too sharp.

Check 5: Dividing by $\sqrt{d}$ keeps softmax tame

Answer all 3 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

What do we divide the dot products by when the key size is $d = 16$? And when $d = 128$?

5.2 MULTIPLE CHOICE

With $d = 64$ and no $\div \sqrt{d}$, what goes wrong?

- A. The scores spread so widely that softmax puts almost everything on one key
- B. Softmax outputs negative numbers
- C. The probabilities no longer add up to 1
- D. All the dot products become 0

5.3 HANDS-ON CODE

Try it yourself. Run the code below. (a) What do the first two `print`s show? (b) Predict the two numbers on the last line before running it. (c) Normalize `y = np.array([10, 20, 30, 40])` with the same one-liner. What do you get, and why?

```
import numpy as np
np.set_printoptions(precision=2)

x = np.array([2, 4, 6, 8])
print(x.mean(), x.std())
print((x - x.mean()) / x.std())

rng = np.random.default_rng(0)
d = 256
q = rng.standard_normal((10000, d))      # 10,000 random queries
k = rng.standard_normal((10000, d))      # 10,000 random keys
dots = (q * k).sum(axis=1)                # 10,000 dot products
print(dots.std(), (dots / np.sqrt(d)).std())
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The mean: the center · rewatch ▶ 0:08-0:27

1.1 4. $16 \div 4 = 4$. One big number pulls the mean up: three of the four numbers are below it.

1.2 15. Every number moves up by 10, so the center moves up by 10 too: $60 \div 4 = 15$.

1.3 False. 4, 6 and 0, 10 both have mean 5, but the second is far more spread out. That is why we need a second number, the standard deviation.

1.4 A. That is the problem the video opens with. Mean and standard deviation measure the drift, so we can undo it (concept 3).

2. Standard deviation: the typical distance · rewatch ▶ 0:28-0:51

2.1 distance from the mean → square → average → take the square root. Distances first, then square them, average the squares, and finish with the square root.

2.2 $\sqrt{2} \approx 1.41$. Mean 3; distances $-2, -1, 0, 1, 2$; squares 4, 1, 0, 1, 4; average $10 \div 5 = 2$; $\sqrt{2} \approx 1.41$.

2.3 C. Every distance from the mean (5) is 0, so there is no spread at all.

2.4 True. The distances double ($-6, -2, 2, 6$), the squares average 20, and $\sqrt{20} \approx 4.47 = 2 \times 2.24$.

3. Normalizing: mean 0, spread 1 · rewatch ▶ 0:52-1:14

3.1 $-1, 1$. Mean 2; distances -1 and 1 ; std = $\sqrt{1} = 1$; so $(1 - 2) \div 1 = -1$ and $(3 - 2) \div 1 = 1$.

3.2 A. 10, 20, 30, 40 is $5 \times (2, 4, 6, 8)$. Subtracting the mean removes any shift and dividing by the std removes any scale, so only the pattern is left.

3.3 $-1, 3$ · mean 1, std 2. $-1 \times 2 + 1 = -1$ and $1 \times 2 + 1 = 3$. The bias sets the new center, the gain sets the new spread.

3.4 False. Only the *typical* distance is 1. The video's own example gives -1.34 and 1.34 .

4. The spread of a sum grows like $\sqrt{n}$ · rewatch ▶ 1:14-1:25

4.1 About 8 and about 16. $\sqrt{64} = 8$ and $\sqrt{256} = 16$.

4.2 B. Four times as many terms: $\sqrt{64} \div \sqrt{16} = 8 \div 4 = 2$.

4.3 False. About $\sqrt{100} = 10$: positive and negative products partly cancel.

4.4 About 27.7. $\sqrt{768} \approx 27.7$ (a simulation gives about 27.8).

5. Dividing by $\sqrt{d}$ keeps softmax tame · rewatch ► 1:26-1:43

5.1 4, and about 11.3. $\sqrt{16} = 4$ and $\sqrt{128} \approx 11.3$.

5.2 **A.** Wide gaps make softmax sharp (F07). Dividing by $\sqrt{d} = 8$ is exactly softmax with temperature $T = 8$.

5.3 **(a) 5.0 2.23606797749979 and [-1.34 -0.45 0.45 1.34] · (b) about 16 and about 1 (15.94 and 1.00 with this seed) · (c) [-1.34 -0.45 0.45 1.34]** (a) The video's numbers. `x.std()` divides by the count n , exactly like the recipe. (b) Each row of `q * k`, summed, is one dot product of 256 random numbers, so the spread is about $\sqrt{256} = 16$; dividing by $\sqrt{d} = 16$ brings it back to about 1. (c) 10, 20, 30, 40 is $5 \times (2, 4, 6, 8)$, and normalizing removes both the shift and the scale, so you get the same numbers.